Distributed Computations in Fully-Defective Networks

Keren Censor-Hillel
Technion, Israel
ckeren@cs.technion.ac.il

Shir Cohen
Technion, Israel
shirco@cs.technion.ac.il

Ran Gelles
Bar-Ilan University, Israel
ran.gelles@biu.ac.il

Gal Sela
Technion, Israel
galy@cs.technion.ac.il

ABSTRACT

We address *fully-defective* asynchronous networks, in which all links are subject to an unlimited number of alteration errors, implying that *all* messages in the network may be completely corrupted. Despite the possible intuition that such a setting is too harsh for any reliable communication, we show how to simulate any algorithm for a noiseless setting over any fully-defective setting, given that the network is 2-edge connected. We prove that if the network is not 2-edge connected, no non-trivial computation in the fully-defective setting is possible.

The key structural property of 2-edge-connected graphs that we leverage is the existence of an oriented (non-simple) cycle that goes through all nodes [Robbins, 1939]. The core of our technical contribution is presenting a construction of such a Robbins cycle in fully-defective networks, and showing how to communicate over it despite total message corruption. These are obtained in a *content-oblivious* manner, since nodes must ignore the content of received messages.

CCS CONCEPTS

• **Computing methodologies** → **Distributed algorithms**; • **Computer systems organization** → *Dependable and fault-tolerant systems and networks*; **Fault-tolerant network topologies**; • **Networks** → *Network protocols.*

KEYWORDS

Fully-defective Networks, Noise Resilience, Robbins' Theorem

ACM Reference Format:
Keren Censor-Hillel, Shir Cohen, Ran Gelles, and Gal Sela. 2022. Distributed Computations in Fully-Defective Networks. In *Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing (PODC '22), July 25–29, 2022, Salerno, Italy*. ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3519270.3538432

1 INTRODUCTION

Faults are a main hurdle in a large variety of distributed systems. Faults manifest themselves in several different manners, ranging from nodes that crash due to malfunctions to environmental disruptions that affect the communication channels connecting distant nodes. In the last few decades, research has focused on developing *fault-tolerant* algorithms, as nodes crashes and channel noise are utterly inevitable. See, e.g., recent books and surveys on fault-tolerant systems [12, 26] and algorithms [4, 33], and references within.

In this work, we consider the case of channel noise within asynchronous distributed networks, where messages communicated between nodes are subject to corruption. When dealing with channel noise, some restrictions must be imposed on its power. Clearly, if noise can affect channels arbitrarily without any restrictions, then it could, for instance, delete all the communication and prevent any non-trivial computation over the network. Previous work either limited the *number* of channels that may suffer (arbitrary) noise [11, 22, 23, 30, 35, 36] or the *total amount* of corruptions (usually, alterations) the channels are allowed to make altogether [2, 8, 18, 24].

Throughout this work, we consider noisy channels that may arbitrarily change the *content* of transmitted messages, but can neither delete nor inject messages. This is known in the literature as alteration noise. Yet, we do not bound the amount of noise nor the number of noisy channels in any way. That is, we ask the following question:

> Can one design fault-tolerant algorithms robust to an *unlimited* amount of corruption on *all* communication channels?

On its surface, the above task seems doomed. However, we answer the question in the affirmative for the large family of 2-edge-connected networks. We further show that if the network is not 2-edge-connected, the noise can destroy any non-trivial computation.

Towards this goal, we develop *content-oblivious* algorithms, that is, algorithms that do not rely on the *content* of communicated messages [8]. Instead, the actions of a node depend on the specific links and the order in which messages are received. In particular, we devise a method that compiles *any* distributed algorithm into a content-oblivious version that computes the same task over 2-edge-connected graphs.

A folklore approach (see, e.g., in [8, 25]) is to send a message along a certain path from u to v to signify a 0 bit, and to send a message along a different path to signify a 1 bit, where the existence of two different paths is promised by the 2-edge-connectivity property. This approach conceals many challenges. First, the edges along these two paths are also edges in paths between other nodes in the network, and so the nodes must somehow be able to associate each such "bit" with its correct origin, in order to be able to decode each original piece of information and avoid mixing up bits of different ones. Second, in order to know where to forward the message to, the nodes need to extract the sender/receiver information from these "bit" messages, yet those might be fully corrupted. Third, some guarantee needs to be obtained on the order in which different 0/1 "bits" arrive at their destination, in order for them to

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PODC '22, July 25–29, 2022, Salerno, Italy
© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9262-4/22/07…$15.00
https://doi.org/10.1145/3519270.3538432

faithfully represent the encoded message, a caveat on which the asynchrony of the network imposes another obstacle.

Before elaborating on how we overcome all these issues and stating our main results, let us explain our setting and noise model in more detail. We abstract the network as a graph $G = (V, E)$ where every node $v \in V$ is a computing device and every edge $e \in E$ is a noisy bi-directional communication channel. Once u sends a message m over some link (u, v), the channel guarantees that after some arbitrary yet finite time, v receives some message $m' \in \{0,1\}^+$. Note that m' may or may not equal m. In other words, the noise over the channel can corrupt the content of any transmitted m into any m', but it cannot completely delete it, nor can it inject new messages. We say that G is a *fully-defective* network if all its channels are noisy in the above manner.

1.1 Our contribution and techniques

Intuition: Content-oblivious encoding with parallel channels. Let us begin with a simple toy example that illustrates some of our techniques. Suppose u and v are directly connected by *two separate noisy channels*, which we name DATA and END. The basic idea is to communicate the information over the DATA channel by sending, "bit-by-bit", a unary encoding of the original message. In order to communicate the end of the unary encoding, a single message is sent on the END channel. Note, however, that timing is crucial: if the message sent on END is received before all the messages sent on DATA reach their destination, the receiver decodes incorrect information. To avoid this confusion, the receiver sends one message over END as an acknowledgment for each received DATA message. The sender waits until all its DATA messages are acknowledged *and only then* sends the termination message on END. Sending the terminating END message has an additional effect: it switches the roles of the nodes. If u is the sender, then after sending the END message it takes the role of the receiver and vice versa. We call the sender at each point the *token holder*.

Main result. Since we do not wish to assume two separate channels between any two nodes, we ask whether they can be replaced with two separate *paths* between any two nodes, i.e., can we constitute reliable communication between any two neighbors in 2-edge-connected graphs?

We answer this question affirmatively and show a method that takes any asynchronous message-passing distributed algorithm π for a noiseless network G, and simulates it over the fully-defective G, given that G is 2-edge connected. By *simulating* we mean that every node has a black-box interface to π through which the node can deliver messages to π and (asynchronously) receive messages to be communicated to some neighbor. The simulation guarantees that, at any given moment, all the nodes behave similarly to some valid execution of π over the noiseless G.

THEOREM 1 (MAIN, INFORMAL). *There exists a simulator for any asynchronous algorithm π such that executing the simulator over a 2-edge connected fully-defective network G simulates an execution of π over the noiseless network G.*

Once we establish that such a simulator even exists, a natural question is, what is the best that could be aimed for in terms of its message overhead? To avoid excessive clutter in the presentation, we delay the complete statement of our main theorem that includes its overhead, to Theorem 2 at the end of this section.

Warm-up: Resilient computations over a simple cycle. To describe our approach for proving Theorem 1, we begin with the much simpler case of *cycle graphs*. In a cycle graph, any node u is connected to only two neighbors. Our goal is to simulate u's communication with its two neighbors over a fully-defective cycle. In this special case, every two neighbors have exactly two separate paths between them: the direct link, and the rest of the cycle. The difference from the two-channel toy example illustrated above is that the paths of each two neighbors intersect the paths of other neighbors and we need to coordinate between the nodes so that each message reaches its correct destination and is interpreted correctly.

We address this difficulty by guaranteeing that only a single node is the sender (token holder) at any given time. All the other nodes are passive and only forward messages along the cycle. In this way, the sender can communicate with its neighbor using both paths of the cycle—one of them, say, the clockwise path, replaces the DATA channel, and the other one, the counterclockwise path, replaces the END channel.

In fact, u can use the same method to communicate with any other node on the cycle, since all the nodes see the same sequence of clockwise and counterclockwise messages. In a sense, u broadcasts information over the cycle, and all the nodes learn this information.

Next, we design a method to change the roles such that another node may become the token holder, i.e., the sender: After forwarding the message initiated by the sender in the END path, the nodes enter a *token delivery* phase. During this phase, a counterclockwise TOKEN message, initiated by the previous token holder, is forwarded along the cycle. When it reaches a node, if the node does not have a message to send, then it forwards the token along by propagating it counterclockwise. Otherwise, it becomes the new token holder and initiates *a clockwise* message forwarded along the entire cycle which denotes the end of the token phase, so all the nodes go back to the stage of interpreting messages as DATA and END.

The above method has one significant drawback. If it takes some time for the nodes to produce a message to send, then the TOKEN message will keep circulating in the cycle, causing many superfluous transmissions. We circumvent this situation of wasteful transmissions by introducing a request mechanism: the token transfer is performed only if some node issues a request, which is done by sending a clockwise REQUEST message. The requesting node can be far away from the current token holder, thus, each node, upon receiving a REQUEST message, propagates it clockwise. We note that several nodes might issue a request at the same time at different locations on the cycle. Eventually, all nodes will have sent and received a REQUEST message, and after it reaches the current token holder, it issues the counterclockwise TOKEN message described above.

This simulator for simple cycles, in which messages are interpreted as DATA, END, TOKEN or REQUEST based only on their direction and order of transmissions, is formally given and proved in Section 3.

Main result: Resilient computations over 2-edge-connected graphs. To apply our approach for simple cycles to more complex graphs, we mimic it over a (not necessarily simple) cycle that goes through all the graph nodes.

Such a cycle needs to be chosen carefully, because of the crucial role that the direction of messages plays in our approach. Robbins's theorem [34] states that any 2-edge-connected graph G is *orientable*. That is, there exists a way to orient all edges in G so that the implied directed graph is strongly connected. This implies that there exists a cycle that goes through all nodes, possibly with multiple occurrences of some of the nodes, where all instances of any edge along the cycle bear the same orientation. We leverage the existence of such a Robbins cycle by mimicking our approach for the simple cycle over the Robbins cycle. To this end, we must first construct a Robbins cycle, as the nodes are unaware of the topology of the network. Then, we need to communicate over the Robbins cycle. Both steps are highly non-trivial and pose many challenges, as we now describe. During the first step of constructing a Robbins cycle we need the nodes to start communicating over partial pieces of the cycle (which are cycles by themselves), for which we need to already use the second step. For this reason, we describe the two steps in reverse order: we begin with describing the second step of communicating over a non-simple cycle, given that each node knows its previous and next neighbors along the cycle for each of its occurrences (Section 4). In the full version [7], we then show the first step of how to construct the Robbins cycle and produce this information.

Second step: Communicating over a non-simple cycle. The input of each node for this step, as will be guaranteed by our construction for the first step, is the previous and next nodes along the cycle for each of its occurrences. These inputs are consistent with some Robbins cycle, so that, in particular, each edge in the cycle has a unique orientation, and a single orientation of the edges is considered by all nodes as the *clockwise* direction.

Mimicking our approach for a simple cycle over a Robbins cycle brings along several challenges. Consider, for instance, the network G and its induced Robbins cycle depicted in Figure 1. Suppose a clockwise message is received at node d along the edge (c, d). Should this message be propagated over the edge (d, e) or over the edge (d, a), or maybe over both? Note that both these options are in the clockwise direction, however, they belong to different segments of the cycle. Further, note that some messages are initiated in an asynchronous manner, e.g., the REQUEST message. Thus, when the node d receives a REQUEST message from node c, it is possible that the request originated at node a and should be propagated to node e or it originated at node e and should be propagated to node a.

We cope with these issues using two separate mechanisms. The first mechanism makes sure that the TOKEN, DATA, and END messages are propagated correctly along the Robbins cycle. This mechanism consists of two main ingredients. First, we guarantee that these three message types are forwarded in a sequential manner, in the sense that the token holder issues the next message among them only after receiving the previous one from the other direction of the cycle. Second, we assure that at any given moment, each node u knows "where the token is", that is, on which segment of the cycle (i.e., between which two occurrences of u) the token resides. Since the token holder is the only node to initiate the above three message types, knowing the relative position of the token holder resolves the above and allows each node to track each message along the Robbins cycle. Indeed, each such message must first arrive from the segment in which the token holder resides and then be propagated by u to the next segment in the respective direction of the cycle. We prove that since at any given moment only one message travels through the cycle, there can be no confusion at u regarding what the message type is, which one of u's occurrences has received a message and where a message should be forwarded to.

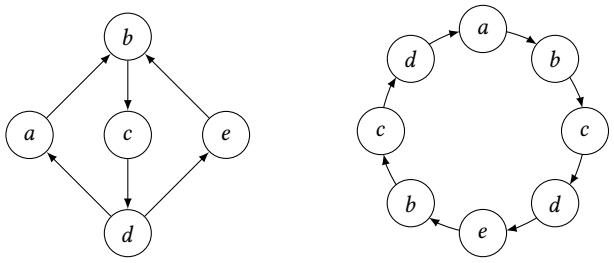


Figure 1: (a) A 2-edge-connected graph G with a Robbins orientation and (b) the resulting Robbins cycle with multiple occurrences per node. The arrows denote the clockwise direction of the cycle.

The second mechanism we employ is for REQUEST messages. These have no pre-specified origin, and they can be initiated by any node and even by multiple nodes at the same time. The mechanism for these messages is as follows. Whenever a node receives a REQUEST message or wishes to initiate one, it sends a clockwise message to *all* of its clockwise neighbors along the Robbins cycle at the same time. Then, the node waits to receive a REQUEST message from each of its counterclockwise neighbors *and only then* it continues with executing the cycle algorithm described above. We prove that this guarantees that all nodes send and receive a request message *regardless* of their position(s) on the Robbins cycle.

First step: A content-oblivious construction of a Robbins cycle. Our Robbins cycle construction follows an *ear-decomposition* technique by Whitney [41], claiming that any 2-edge-connected graph G can be decomposed into edge-disjoint parts, $G = C_0 \cup E_0 \cup E_1 \cup \cdots \cup E_k$, where C_0 is a simple cycle, and for any $0 \leq i \leq k$, E_i is an *ear*—a simple path or cycle whose endpoints belong to $C_0 \cup E_0 \cup \cdots \cup E_{i-1}$. Following Whitney's work, we iteratively decompose a 2-edge-connected graph G into some $C_0, E_0, \ldots, E_k$, part by part, and combine them into a Robbins cycle. The main obstacle we face is that our construction must be content-oblivious and cannot rely on the content of messages sent by the nodes.

The first stage of our construction is performing a DFS-like search starting from a specified *root* node. The DFS search progresses by sending a message (*a DFS-token*) sequentially, i.e., each node propagates this message to one of its unexplored adjacent edges. This DFS-token message propagates through the network until it reaches the root node again. At this stage, the path the DFS-token has taken defines a cycle C_0.

The key challenge in this stage is that the DFS-token might reach a node u twice before reaching the root, causing the DFS to "get stuck", e.g., if $deg(u) = 3$. We overcome this pitfall by insisting on C_0 being a simple cycle that starts and closes at the root. If some $u \neq root$ receives the DFS-token for the second time, it sends that message back on the same edge on which it was received. This

has the effect of "backtracking" that edge, excluding it from the constructed cycle. Nodes that backtrack all their adjacent edges go back to their initial state and are added to the Robbins cycle later.

Once C_0 is established, the nodes on it switch to the second stage, in which they use our resilient communication approach of the above second step, in order to coordinate exploring further ears. One node on C_0 that has adjacent edges that do not belong to C_0 gets selected to initiate another DFS-like search, which again propagates in G until reaching a node on C_0, possibly different from the initiator. The path the DFS-token takes defines the ear E_0. Then, the nodes on C_0 and E_0 jointly coordinate to form a new non-simple cycle C_1 that includes all the edges in C_0 and E_0. The nodes on C_1 switch to communicate over this cycle using the above resilient communication of the second step. The nodes iterate this process, until a Robbins cycle is formed. A crucial aspect of these iterations of adding ears is that much coordination is required among the nodes for switching in a timely manner from communication on C_i to communication on C_{i+1}. The technical specification of this mechanism is given in the full version [7].

We emphasize that the nodes do not know $|V|$, and hence they do not know when a Robbins cycle is already formed, i.e., when each node already appears on the current C_i at least once. Instead, they keep adding edges to the constructed cycle, until no node has an adjacent edge that is not in C_i, which is a state they can detect. At that point, the construction ends.

Putting it all together. With the above two steps, our result can now be formally stated. Given any 2-edge-connected fully-defective network G and an asynchronous algorithm π designed to work on the noiseless G, we show how to compute π over the fully defective G by first constructing a Robbins cycle C on G using a resilient content-oblivious algorithm, and then simulating π over the Robbins cycle C in a resilient content-oblivious manner.

Theorem 2 (main). *There exists a simulator for any asynchronous algorithm π, such that executing the simulator over a 2-edge connected fully-defective network G simulates an execution of π over the noiseless network G.*

The simulator has a pre-processing phase that construct a Robbins cycle C (which depends only on G) and an online phase that simulates the communication of π over C. The pre-processing step communicates $\mathrm{CC}_{\mathsf{init}} = |C|^{O(1)}$ bits. In the online phase, any message m communicated by π is simulated by communicating $\mathrm{CC}_{\mathsf{overhead}}(m) = O(|C| \cdot |m| + |C| \log |V|)$ bits.

We note that, in the worst case, $|C| = O(|V|^3)$. We do not strive to optimize the polynomial overhead of our schemes, as their mere existence is the focus of this paper. Nevertheless, unary encoding as explained above imposes an exponential overhead in the length of the message. In the full version [7] we offer a binary encoding method that reduces the communication complexity to the polynomial terms stated above.

Impossibility result. We complement the above result and show that if G is not 2-edge-connected, then there is no way to conduct non-trivial computations over a fully-defective G. To this end, we prove the following impossibility for *two-party computation* over a fully-defective channel. The impossibility for a non 2-edge-connected G follows since it contains a bridge, and we can reduce the two sides of the bridge to the two-party case.

Theorem 3. *Fix a non-constant function $f(x, y)$. No two-party deterministic algorithm that gives output or terminates can compute f over a fully-defective channel.*

The theorem requires the nodes to either terminate or irrevocably give an output. Note that the above theorem differs from the famous *two generals coordinated-attack* impossibility [21], since our noise model does not allow deleting messages. See the full version [7] for complete details.

1.2 Related work

There are two common ways to deal with channel corruptions. One is by adding redundancy, i.e., coding the information, an approach that is known in the literature as *Interactive Coding*. The other is by diverting the communication so it would not pass through corrupted edges, which are known as *Byzantine edges*.

We review some related work in these areas, but we stress that neither approach can be used in fully-defective networks: Interactive coding must assume some bound on the errors, either per channel or globally, while solutions for networks with Byzantine edges must assume a bound on the number of noisy channels.

Interactive coding was initiated by the seminal work of Schulman [32, 38, 39], see [15] for a recent survey on this field. In this setting, communication channels either suffer from stochastic noise [3, 6, 17, 19, 32] or from some bounded amount of adversarial noise. E.g., if limiting the overhead of the coding scheme to be linear, [18, 19, 24, 25, 27] develop schemes resilient to up to a fraction $O(1/|E|)$ of the total communication. Without any restriction on the overhead, schemes can cope with noise up to a fraction $O(1/|V|)$ of the total communication, and such a fraction is shown to be maximal [25]—otherwise, the adversarial noise could completely corrupt all the outgoing communication of the node that communicates the least. The above works assume synchronous networks. Censor-Hillel, Gelles, and Haeupler [8] developed the first coding scheme for *asynchronous* networks that suffer from up to a fraction $O(1/|V|)$ of adversarial noise. Communication with an unbounded (yet, *finite*) amount of noise was examined in [1, 9, 16] for the two-party case and in [2] for the multiparty case. In a work by Efrmenko, Haramaty, and Kalai [13], the noise model is similar to the one we consider here in the sense that it can corrupt *the content* of messages but not their existence. However, the amount of bit-corruptions in [13] (measured as the edit distance between sent and received messages) is bounded to a constant fraction out of the entire communication. Furthermore, their work considers only two parties.

Networks with Byzantine edges do not restrict the amount of noise per link, and even allow insertion/deletion errors, but allow only a bounded number of links to be noisy. In asynchronous settings, Fisher, Lynch, and Paterson [14] exclude the existence of consensus algorithms when a single node may crash, or equivalently, when all the links connected to some single node may crash. In synchronous networks, certain tasks are also impossible with arbitrary link failures [21, 37]. On the other hand, Santoro and Widmayer [35] considered distributed function evaluation when (a large number of) links suffer either corruptions, insertions, deletions, or

their combination. In a sense, the synchrony guarantee allows simpler solutions, e.g., encoding information via the time in which messages are sent. Pelc [30] shows that if the number of Byzantine links is bounded by f, robust communication is achievable only over graphs whose edge-connectivity is more than $2f$. This is also implied by the work of Dolev [11]. Additional works [5, 10, 20, 31, 40] consider the case of mixed node and link failures.

Recent work by Hitron and Parter [22, 23] gives a compiler that turns any algorithm in the noise-free setting into an algorithm that works correctly even if the adversary controls f edges in a $(2f+1)$-edge-connected network. The above is for the synchronous Congest setting. Their approach is to construct a family of low-congestion cycle-covers (see also [28, 29]), which are structures in which for every edge (u, v), there are at least $f + 1$ cycles that contain no adversarial edges. We stress that low-congestion cycle-covers do not seem to be helpful for our setting: Even if we were promised only *two* cycles that share a *single* edge, it is not clear how to communicate over them in a way that distinguishes one from the other.

2 PRELIMINARIES

Notations. We use $a \| b$ or $a \cdot b$ for the concatenation of a and b. For a positive integer $k \in \mathbb{N}$ and a string b, we let $b^k = b \cdot b \cdot \cdots \cdot b$ denote b concatenated to itself for k times; $b^0 = \epsilon$ is the empty string. For a string b and an integer $0 \leq i \leq |b| - 1$, we let b_i denote the i-th bit of b, i.e., $b = b_0 b_1 \cdots b_{|b|-1}$.

Networks and protocols. A protocol π over an undirected network $G = (V, E)$ with $n = |V|$ nodes is an asynchronous event-driven distributed algorithm, in which nodes conduct some computation by sending messages to their neighbors in G (for simplicity, we assume only deterministic algorithms in this paper). Upon the reception of a message, π instructs the recipient node what message(s) to send next, as a function of the node's input and all the messages it has received so far. Specifically, each node v begins with a private input x_v (which may be empty), and knowledge of the IDs of its neighboring nodes, $N(v) = \{u \mid (u, v) \in E\}$ (we can remove this assumption, see the full version [7]). According to the input to v, π generates messages to send to zero or more of v's neighbors (possibly different messages to different neighbors). Afterwards, the protocol behaves in an event-driven manner, i.e., nodes act only upon receiving messages: whenever a node v receives a message, it performs some computation and produces messages designated to zero or more of its neighbors. We impose no assumption on the computation time of π except that it is finite. We additionally assume a preselection of one designated node (which will function as a root node in our Robbins cycle construction), and assume that every node knows whether it is the designated node.

Communicating a message over some link of G takes arbitrary positive finite time. Channels are *not* assumed to be FIFO. Incoming messages are kept in an incoming buffer until processed by the node.

The *protocol's transcript* τ of a given execution, is the sequence of messages sent and received during the execution. Each item in τ indicates the message sent or received, the sending or receiving node and the link on which the message was communicated. Events that happen in different nodes at the same time are assumed to be ordered in some arbitrary order. The *local transcript* τ_v of a node v, is the ordered sequence of messages sent and received by v. Note that τ_v can be derived from τ as the sub-sequence in which v is the sending or receiving node.

We say that π *gives an output* if every node eventually writes an output to its write-only output register. This action is irrevocable. If needed, the node may remain active and send/receive messages; that is, we do not require termination, but our result also applies to protocols that terminate. We say that the protocol has reached *quiescence* at some time, if no message is still in transit and from that time on, no new messages are sent over the network.

Fully-defective networks and noise-resilient simulations. We work in networks with noisy channels exposed to alteration noise, which can corrupt the *content* of any message communicated over any channel. That is, once a message $m \in \{0, 1\}^+$ is sent over some link, the received message may be any $m' \in \{0, 1\}^+$. However, the noise *cannot* completely delete a message nor can it inject a message on a link in which no m was sent. We stress that, except for inserting and deleting messages, the noise has no restrictions at all. In particular, it can apply to all channels and corrupt all messages in a given execution. We call networks that suffer noise as specified above *fully-defective* networks. Equivalently, one can think about such a network as one in which nodes communicate only by means of sending pulses to their neighbors, which could be the case, for instance, when the nodes have very basic communication hardware.

A *noise-resilient simulator* designed for a noiseless network $G = (V, E)$ is a protocol $\hat{\pi}$ which is given as an input an asynchronous black-box interface to some π. When $\hat{\pi}$ is executed on a fully-defective network G, it produces for each node $v \in V$ a string $\hat{\tau}_v$, such that there exists some execution of π over the noiseless network G that generates a transcript τ, for which $\tau_v = \hat{\tau}_v$ for each node v. We allow a simulator to perform pre-processing before simulating π. We define $\mathrm{CC}_{\mathsf{init}}$ to be the communication complexity in bits of the simulator during the pre-processing, and $\mathrm{CC}_{\mathsf{overhead}}(m)$ to be the communication complexity for simulating the delivery of a message m. Note that CC accounts only for the length of *sent* messages, even if later their content is corrupted by the noise.

Distributed representation of cycles. A (directed) cycle can be represented in a distributed network in two manners: *locally* and *globally*. A local representation of some cycle C means that every node on C knows its two neighbors on the cycle along with their respective direction, clockwise or counterclockwise, usually held in the local variables *next* and *prev*, respectively. In case C is not a simple cycle, then every node knows its clockwise and counterclockwise neighbors for each of its occurrences on C. This information is consistent across all nodes in the sense that an outside observer who follows the neighbors and directions of each node would see a consistent directed cycle.

A global representation of a directed cycle means that every node $v \in C$ holds the string $C = (v_1, v_2, \ldots)$ of the IDs of the nodes on C in their clockwise order.

3 SIMULATING COMPUTATIONS OVER A FULLY-DEFECTIVE SIMPLE CYCLE

As discussed in Section 1, we can establish a resilient connection between two nodes connected by two separate links, sending *contentless* messages between them, which we will call *pulses* throughout

this paper. Our goal is to implement this idea for any two nodes in a 2-edge-connected graph, since in such a graph any two nodes are connected by two separate *paths*. As a stepping stone, in this section we consider the special case of *simple cycles*.

Theorem 4 (A simulator for a simple cycle). *There exists a noise-resilient simulator for any asynchronous protocol π and any fully-defective simple cycle G in which each node knows its clockwise and counterclockwise neighbors. The simulator features* $\mathrm{CC}_{\mathsf{init}} = 0$ *and* $\mathrm{CC}_{\mathsf{overhead}}(m) = O(|V| \cdot |m| + |V| \log |V|)$ *pulses.*

Let G be a simple cycle on $V = \{v_i\}_{0 \le i \le n-1}$ with $E = \{(v_i, v_{i+1})\}_{0 \le i \le n-1}$, where indices are taken mod n. The main idea is to imitate the two-channel idea described in Section 1 above over the cycle. That is, suppose v_i wishes to send a message to its neighbor v_{i+1}. We can think of the link (v_i, v_{i+1}) as the DATA channel, and on the *path* $v_i, v_{i-1}, v_{i-2}, \ldots, v_{i+1}$ as the END channel. For this to work, all the nodes beside v_i and v_{i+1} need to simply forward each pulse they receive along the same direction. However, the above description supports only a single fixed sender and a single fixed receiver. Thus, we need a method that allows different nodes to become the sender. For this we use a token mechanism, where only a single node holds the token at any given time.

Our simulator can be split into two separate phases per message transfer: the first one is the *token phase* which handles transferring the token between the nodes, and the second one is the *data phase* that handles communication between the current token-holder and the rest of the nodes.

The token phase works as follows. At the start, there exists only a single token holder. During the token phase, pulses carry one out of two possible meanings: either they are a REQUEST pulse or a TOKEN pulse. The meaning of a pulse is dictated by the direction in which the pulse progresses along the cycle: REQUEST is a clockwise pulse while TOKEN is a counterclockwise pulse. A node that wishes to obtain the token issues a REQUEST pulse. Every node that receives such a REQUEST pulse, propagates it in the same direction, unless it has already sent a REQUEST pulse previously in this phase, so eventually every node sends and receives *a single* REQUEST pulse.

Upon receiving a REQUEST pulse, the current (single) token holder releases the token by sending a counterclockwise TOKEN pulse. This pulse propagates along the cycle until it reaches one of the nodes which requested the token. A node that receives the TOKEN pulse and wishes to become a token holder does not propagate the TOKEN pulse but instead sets itself as the new token holder. Then, the new token holder switches to its data phase and begins sending clockwise pulses, which are interpreted as DATA pulses. The first of these pulses propagates throughout the entire cycle and informs all the other nodes that the token phase has completed. This first pulse cannot be confused with a REQUEST pulse since we guarantee that every node sends and receives exactly a single REQUEST pulse in each token phase. In other words, the second clockwise pulse received during a token phase must be a DATA pulse, which triggers its recipient to switch to its data phase.

In the data phase, the token holder delivers its message via a unary coding. That is, it sends a number of clockwise DATA pulses that equals the length of the unary encoding of the information. Each node other than the token holder forwards each received DATA pulse clockwise, so these pulses propagate along the cycle until they reach the token holder back from the other side of the cycle. Then, the token holder sends a single counterclockwise END pulse that signals the end of the message and the end of the data phase. Note that once the token holder receives the END pulse from the other direction, all nodes know that the data phase is over, and are back in the token phase. Note also that due to the asynchrony, nodes that already moved to the next token phase might send a REQUEST pulse before the END pulse arrives at the token holder. Our design promises that these REQUEST pulses are not confused with pulses of the current data phase: END pulses are sent in the other direction, and as for DATA pulses—the token holder does not proceed to sending a REQUEST pulse before it receives the END pulse of the data phase, so REQUEST pulses of the new token phase can only reach nodes that have already received the END pulse for this phase and therefore do not interpret them as additional DATA pulses.

A *phase* is a local concept, in the sense that each node runs a specific data or token phase in any given time, and different nodes might be in different phases in a certain time. We denote each token phase and its subsequent data phase an *epoch*. An epoch is a local concept too, viewed by each node according to the phase it is currently running. Different nodes might be in different epochs in a certain time: some nodes might already send a REQUEST pulse in the new epoch while others have still not received an END pulse for the previous epoch.

Formal description of the simulator. We now formally describe our simulator over fully-defective simple cycles, where each node is given the identities of its clockwise and counterclockwise neighbors. Our simulator receives as an input an asynchronous protocol π for noiseless communication channels. Messages to be sent are generated by π, and any message received by a node in our simulator is delivered and processed by π. Our simulator thus treats π as an asynchronous black box that interfaces with the simulator by sending and receiving messages, internally at each node. We stress that π's actions take finite arbitrary time unknown to and independent of the simulator algorithm.

Our simulator appears in Algorithms 1(a) and 1(b). All nodes begin executing the token phase (Algorithm 1(a)). Each node u has an internal $isTokenHolder_u$ variable that indicates whether it is the token holder. Moreover, each node u keeps a queue Q_u of messages generated by π, which should be broadcast over the cycle. Messages in Q_u are of the form (m, u, v), where m is a message that π instructs u to send to v. At the onset, $isTokenHolder_u$ is True for a single node, and each Q_u is empty. When π gives an output, the respective node gives the same output in the simulator but keeps executing the communication algorithm over the cycle. If in a certain time all the queues $\{Q_u\}$ are empty and remain empty, then the simulator stops sending messages and reaches quiescence.

The simulator is content-oblivious, and as such it communicates by sending pulses (content-less messages). Note that in our algorithms we write next to each pulse its meaning (DATA, END, REQUEST, TOKEN), however, this is only for the analysis; the nodes assign this meaning according to their current state and the clockwise/counterclockwise direction of the pulse, and not by the pulse content, which is ignored.

Algorithm 1(a) A simulator for simple cycles: token phase (node u)

Init: A single node has *isTokenHolder* = True. Node u holds a (possibly empty) input x_u for π.
Handling messages sent by π: During the execution of the algorithm, node u enqueues to Q_u any new message π asks u to send, in the form (message, source, destination). The actions of π occur in parallel to the execution of this algorithm.

1: **wait until** Q_u is not empty or a clockwise REQUEST pulse is received
2: send a clockwise REQUEST pulse
3: **if** no clockwise REQUEST pulse was received **then** wait until receiving a clockwise REQUEST pulse **end if**
4: **if** $isTokenHolder_u$ **then**
5: $isTokenHolder_u \leftarrow$ False
6: send a counterclockwise TOKEN pulse
7: **end if**
8: **wait until** receiving a pulse
9: **if** the pulse is a counterclockwise TOKEN pulse **then** ▷ Else, the pulse is a clockwise DATA pulse
10: **if** Q_u is not empty **then**
11: $isTokenHolder_u \leftarrow$ True
12: **else**
13: forward the counterclockwise TOKEN pulse
14: **go to** Line 8
15: **end if**
16: **end if**
17: continue with Algorithm 1(b)

Algorithm 1(b) A simulator for simple cycles: data phase (node u)

18: **if** $isTokenHolder_u$ **then**
19: dequeue a message from Q_u, denote it by (m, u, v) and let 1^d be its unary encoding
20: send d clockwise DATA pulses
21: **wait until** receiving d clockwise DATA pulses
22: send a counterclockwise END pulse
23: **wait until** a counterclockwise END pulse is received
24: **else**
25: forward any received clockwise DATA pulse **until** receiving a counterclockwise END pulse
▷ Including the DATA pulse received during the preceding token phase
26: let *count* be the number of received clockwise DATA pulses
27: decode 1^{count} as the unary encoding of (m', u', v')
28: **if** $u = v'$ **then** deliver m' to π (as if received from u') **end if**
29: forward the counterclockwise END pulse
30: **end if**
31: continue with Algorithm 1(a)

3.1 Analysis

We set some notation for the analysis of Algorithm 1. Let '1' indicate a pulse sent clockwise, and let '0' indicate a pulse sent counterclockwise. Recall that an *epoch* is the execution of consecutive token and data phases. We say that a node has completed its k-th epoch once it has executed Line 31 for the k-th time. Let T_k be the k-th node to have set its $isTokenHolder_u$ to True in Line 11, whereas T_0 is the node whose $isTokenHolder_u$ variable is initialized to True. (We will show that T_k sends, in its k-th epoch, the k-th simulated message in the system.) Let s_k, for $k \geq 1$, be the time in which T_k sets its *isTokenHolder* $\leftarrow$ True ($s_k = \infty$ if T_k is undefined). Finally, let t_k be the time in which T_k completes its k-th epoch ($t_k = \infty$ if T_k is undefined or never ends the k-th epoch). Let $s_0 = t_0 = 0$. Let $[u \curvearrowright v]$ denote the clockwise path from u to v along the cycle including both ends, and similarly $[u \curvearrowleft v]$ denotes the counterclockwise path from u to v. In the special case of identical endpoints, $[u \curvearrowright u]$ denotes the path through the whole cycle. To exclude an endpoint, we use a round bracket, e.g. $[u \curvearrowright v)$ denotes the clockwise path excluding v; the path can be empty, i.e., $(u \curvearrowright v)$ for u, v neighbors.

Our analysis is based on the following technical lemma, which provides us with three important properties satisfied by Algorithm 1 in every epoch: (1) **progress,** which says that as long as there is a message to send, the next epoch will eventually start and complete; (2) **single token holder,** which says that at most a single node holds the token at any moment (there is no such node during the time in which the token is being passed), and T_k is the only one to hold it during the data phase of the k-th epoch; and (3) **global consistency,** which says that in any given epoch k, exactly one message is being communicated—sent by T_k and received by all other nodes, and the pattern of pulses every node sends has a distinct structure. We now formalize these ideas as follows.

Lemma 5. *Consider an execution of Algorithm 1 and consider any $k \geq 1$, for which $t_{k-1} < \infty$. If from time t_{k-1} and forward, all queues $\{Q_v\}_{v \in V}$ are always empty, then $t_k = \infty$. Otherwise, the following hold:*

(1) ***Progress:*** *All nodes eventually complete their k-th epoch. In particular, $t_k < \infty$, and at time t_k, all nodes have already processed the END pulse (of epoch k) but have not yet passed Line 8 in epoch $k+1$ (they are either waiting in Lines 1 or 3 for a REQUEST pulse, or waiting in Line 8 for either a DATA or a TOKEN pulse).*

(2) ***Single token holder:*** *It holds that $t_{k-1} < s_k < t_k$. At each moment in (t_{k-1}, t_k), there is at most a single node for which isTokenHolder = True. More specifically, within this time frame, the token is passed as follows: the node T_{k-1} releases the token at some time in $[t_{k-1}, s_k)$ and the node T_k is the next node that gains the token at time s_k. The node T_k (solely) holds the token in $[s_k, t_k]$.*

(3) ***Global consistency:*** *There exist integers $d_1, \ldots, d_k > 0$ and for any $u \in V$ there are $b_1^u, \ldots, b_k^u \in \{0, 1\}$, such that when the node u completes its k-th epoch, its sent transcript (the overall pulses sent so far by u) is $P_{u,k} \triangleq 10^{b_1^u} 1^{d_1} 0 \cdot 10^{b_2^u} 1^{d_2} 0 \ldots 10^{b_k^u} 1^{d_k} 0$.*
In addition, the message each node decodes and processes (Lines 26–28) in its k-epoch is the unary decoding of 1^{d_k}, which is the message sent by T_k (Line 19) in its k-th epoch.

Proof. We prove the statement by induction on the epoch number k. We start with proving the base case, $k = 1$. The proof for the general case is very similar, and we defer the differences to the full version [7]. The analysis follows the progress of the protocol and

shows that each pulse sent with a certain meaning (i.e., DATA, END, TOKEN, REQUEST) is correctly interpreted by its recipient.

Base Case, $k = 1$**.** Note that $t_0 = 0$, hence, $t_{k-1} < \infty$. All the nodes begin by executing Algorithm 1(a), with a single node T_0 having *isTokenHolder* = True. While all nodes have empty queues Q_v, they all wait in Line 1 and thus, if the queues remain empty indefinitely, we have $t_1 = \infty$.

Otherwise, at some time there is at least one node u that enqueues to Q_u a message to send. Each such u sends a REQUEST pulse in Line 2 and waits to receive a REQUEST pulse in Line 3, unless it has already received such a pulse in Line 1. As there is at least one such node, at least one REQUEST pulse is sent. The rest of the nodes first wait to receive a REQUEST pulse and then forward it. It follows that all nodes eventually receive and send a single REQUEST pulse. Let us denote by $\tilde{P}_u$ the partial transcript of a node u at the "current" time (which evolves with the proof), then $\forall u \in V$, we have $\tilde{P}_u = 1$ after sending the REQUEST. To prove Property (3), we keep track of the partial transcript $\tilde{P}_u$, recording the pulses sent by each node.

After sending and receiving a REQUEST pulse, any node $u \neq T_0$ waits to receive another pulse (Line 8). The node T_0 sets $isTokenHolder_{T_0}$ = False, sends a counterclockwise TOKEN pulse (Line 6) and then waits for another pulse like all other nodes. The TOKEN pulse triggered by T_0 propagates counterclockwise until it reaches a node v with a non-empty Q_v, which must exist. The node v subsequently sets $isTokenHolder_v$ to True (Line 11). Thus, by the above definitions, we get that $T_1 = v$ and s_1 is the time when v executes Line 11. Note that T_1 might get the TOKEN pulse before getting a REQUEST pulse, in which case it delays its actions until a REQUEST pulse is received. This has no effect on the proof.

In case $T_1 \neq T_0$, at time s_1, all the nodes on $[T_0 \rightharpoonup T_1)$ have sent a TOKEN pulse and are now waiting for a DATA pulse (Line 8) that would switch them to their data phase of epoch $k = 1$. The nodes on $(T_0 \rightharpoondown T_1)$ could be in two possible stages: either they are still waiting for a REQUEST pulse (Lines 1 or 3) as described above, or they are waiting in Line 8. Hence at time s_1, every node $u \in [T_0 \rightharpoonup T_1)$ has $\tilde{P}_u = 10$, while every node $u \in (T_0 \rightharpoondown T_1]$ has $\tilde{P}_u = \epsilon$ if it has not yet sent a REQUEST pulse, or $\tilde{P}_u = 1$ otherwise. Eventually, perhaps at a different time per node, each node u thus reaches the partial transcript $\tilde{P}_u = 10^{b_1^u}$ with $b_1^u \in \{0, 1\}$ being the indicator of whether u has sent a TOKEN pulse (namely, whether $u \in [T_0 \rightharpoonup T_1)$).

In the special case where $T_1 = T_0$, at time s_1, the token has just reached back at T_1; all nodes have sent a TOKEN pulse, and all nodes but T_1 are now waiting for a DATA pulse (Line 8) that would switch them to their data phase of epoch $k = 1$. Hence at time s_1, every node u has the partial transcript $\tilde{P}_u = 10^{b_1^u}$ with $b_1^u = 1$ indicating that u has sent a TOKEN pulse.

When the node T_1 switches to the data phase (Algorithm 1(b)), its queue Q_{T_1} is non-empty and so it sends $d \geq 1$ clockwise DATA pulses (Line 20). We define $d_1 = d$. These DATA pulses propagate clockwise through all nodes, after the first received DATA pulse in each node but T_1 triggers it to switch to its data phase, after it has previously received a REQUEST pulse. Note that each such node must have received a REQUEST pulse before it receives the first DATA pulse. This is because its counterclockwise neighbor, who sends the DATA pulse, moves to the data phase only after it has sent a clockwise REQUEST pulse.

Once a node $u \neq T_1$ is in its data phase of epoch $k = 1$, it records each received DATA pulse. The node propagates the pulse clockwise and eventually the pulse arrives back at T_1. Thus, since T_1 sends d_1 DATA pulses, after propagating them, each node u has $\tilde{P}_u = 10^{b_1^u} 1^{d_1}$. Once the d_1 clockwise pulses reach back at T_1, and only then, it issues a counterclockwise END pulse (Line 22). T_1 does not generate nor does it propagate any additional pulses before receiving the propagated END from the other side of the cycle. This implies that any $u \neq T_1$ receives exactly d_1 clockwise DATA pulses followed by a counterclockwise END pulse. Upon receiving the END pulse, u processes the message 1^{d_1} (Lines 26–28) and forwards the END pulse (Line 29). It then completes its k-th data phase and its k-th epoch, with $\tilde{P}_u = 10^{b_1^u} 1^{d_1} 0$. The node T_1 also has $\tilde{P}_{T_1} = 10^{b_1^{T_1}} 1^{d_1} 0$ when it receives the END pulse and switches to the next token phase, at time t_1. At that time, all the other nodes have already processed the END pulse. This proves the first part of Property (1).

Next, we need to prove that at time t_1, none of the nodes has passed Line 8. In order for a node to pass Line 8, it must be the case that *after* the node has switched to the token phase, it has received a REQUEST pulse followed by one additional pulse (in any direction). We argue this cannot happen. Indeed, at the time where some node v receives the END pulse and switches to its second token phase, only the nodes in $(T_1 \rightharpoonup v]$ have received the END pulse and only these nodes have switched to their (second) token phase. In their token phase, they may or may not have sent a clockwise REQUEST pulse by this time. Thus, only nodes in $[T_1 \rightharpoonup v)$ might have *received* a REQUEST pulse. However, it is impossible that they received an additional pulse by time t_1, as we next show.

Each node in $(T_1 \rightharpoonup v)$ that has received a REQUEST pulse is waiting to receive another pulse (Line 8) and is not generating any pulse. The node T_1, if receiving a REQUEST pulse, does not process it and does not send a REQUEST pulse before receiving an END pulse in Line 23. It also never sends a pulse in the counterclockwise direction before receiving its END pulse back. Furthermore, each node in $(T_1 \rightharpoondown v)$ (for $v \neq T_1$) is still executing Line 25, so it only forwards pulses and never generates pulses. Finally, v has just started its token phase and is waiting to receive a REQUEST. We conclude that no additional pulse (beyond the REQUEST pulse, if sent) can be received by the nodes in $(T_1 \rightharpoonup v)$. This holds for any v at the time it transitions to its second token phase. It thus holds for all nodes at time t_1, when the END pulse eventually reaches back at T_1.

Next we prove Property (2) based on the above description of the first epoch. At the onset (at time t_0), T_0 is the only node with $isTokenHolder_{T_0}$ = True. As mentioned above, T_0 sets $isTokenHolder_{T_0}$ = False and sends a TOKEN pulse during its token phase. The propagated TOKEN pulse is the one that triggers T_1 to set $isTokenHolder_{T_1}$ = True later, at time s_1. Thus, it is clear that $s_1 > t_0 = 0$, and that T_0 releases the token before s_1 and T_1 becomes a token holder at s_1. Later, at time t_1, the node T_1 completes the first epoch, hence, $t_1 > s_1$. The node T_1 does not set *isTokenHolder* = False during the time frame $[s_1, t_1]$, and it remains to show that it is the only token holder throughout this time frame.

It is clear that no node in $(T_0 \rightharpoonup T_1)$ has set itself as a token holder as otherwise, that node would have been the node we indicate as T_1. After time s_1, no more TOKEN pulses are sent in the first token phase. Further, T_1 sends a clockwise DATA pulse that transitions all other

nodes into their data phase. Thus, no node besides T_1 can execute Line 11 and set *isTokenHolder* = True in this token phase. As for the second token phase, each node that reaches it before t_1 does not pass Line 8 before time t_1, as we have shown above, thus in particular, it does not receive a TOKEN pulse and does not reach Line 11.

Finally, we prove Property (3), that is, that all nodes reach a global consistency regarding the sent message of the first epoch. This follows from the above analysis: As we argued, at the time some node u completes its first epoch, it holds that $\tilde{P}_u = 10^{b_1^u} 1^{d_1} 0$. The part 1^{d_1} corresponds to the d_1 DATA pulses initiated by T_1, which form the encoding of the message communicated in this epoch. This completes the proof of Property (3).

Induction Step. Most of the above proof holds as is for $k > 1$, if we replace s_1, t_1, T_1 with s_k, t_k, T_k, etc. We complete the details in the full version [7]. □

The full version [7] shows that Lemma 5 implies the correctness of the simulator, and analyzes its overhead. It then discusses ways to improve the obtained complexity. Together, these prove Theorem 4. Let us point out a couple of remarks about our simulation.

Remark 1. **No starvation:** Our proof shows that as long as some u has a message to send, *some message* will be sent during the next epoch. Since the TOKEN pulse travels counterclockwise sequentially in the cycle, there can be at most $n - 1$ epochs until u becomes the token holder. Thus, our simulator actually satisfies the stronger notion of *no starvation*.

Remark 2. **Broadcast:** We note that by design, our simulator offers an additional *broadcast* operation. That is, a node can send a message whose destination is all other nodes. To provide this functionality, we utilize the fact that every message arrives at all nodes, regardless of its original destination. To broadcast a message m, a node simply fixes its destination to be $*$. Each node that decodes a message delivers it to π if its destination is either that node (as before) or $*$. We will use this feature in our Robbins cycle construction.

Remark 3. **Reducing the communication via binary encoding:** Encoding each message via a unary encoding leads to a pulse overhead that is exponential in the message size: $\mathrm{CC}_{\mathrm{overhead}}(m) = \mathrm{poly}(n, 2^{|m|})$, with $n = |V|$. In the full version [7] we show how to send messages over a simple cycle via a binary encoding of the message. This binary encoding leads to a much improved communication complexity of $\mathrm{CC}_{\mathrm{overhead}}(m) = O(n \cdot |m| + n \log n)$.

4 SIMULATING COMPUTATIONS OVER FULLY-DEFECTIVE 2-EDGE CONNECTED NETWORKS

We show how to perform resilient computations over a 2-edge-connected fully-defective network, given a Robbins cycle.

THEOREM 6 (A SIMULATOR FOR A ROBBINS CYCLE). *Let C be a Robbins cycle (over G) and let each node know its clockwise and counterclockwise neighbors for each of its occurrences on C. There exists a noise-resilient simulator over the fully-defective G for any asynchronous protocol π. The simulator features $\mathrm{CC}_{\mathrm{init}} = 0$ and $\mathrm{CC}_{\mathrm{overhead}}(m) = O(|C| \cdot |m| + |C| \log |V|)$ pulses.*

Let G be a 2-edge-connected graph, and assume the nodes are given a Robbins cycle C, namely, a directed cycle that passes through each vertex at least once, and that does not use any edge in both of its directions. (We stress that this assumption is later removed by showing how to construct the Robbins cycle from scratch.) As a node u may appear in C more than once, we denote by k_u the number of its occurrences on C. The initial knowledge of each node u about C is the value of k_u and its clockwise and counterclockwise neighbors along the cycle. That is, every node u knows the nodes $prev_{u,i}$ and $next_{u,i}$ for every $0 \leq i \leq k_u - 1$, such that the nodes along the cycle C correspond to the *prev* and *next* variables of all nodes in a consistent manner. We refer to the sequence of nodes between two successive occurrences of u on C (including the ending occurrence of u) as a *segment* $(u \rightarrow \cdots \rightarrow u]$.

The high level approach for the algorithm builds upon Algorithm 1 of the simple cycle, with pulses forwarded across the Robbins cycle C. By mimicking the protocol for the simple cycle, u views each of its occurrences along C as a different node along the cycle. Accordingly, when a node u is the token holder, it has exactly one occurrence on C which is associated with holding the token, and when we refer to a token holder in this section, we refer to that precise occurrence. When any node u forwards a pulse in some direction, it forwards it to the node along C that follows its occurrence that received the pulse. There are several challenges in this generalization.

Challenge 1: Edge repetition along C. Perhaps the main challenge is for u to keep track of its occurrences and distinguish between them: it could be that multiple occurrences of u have the same incoming edge. Still, this node needs to be able to associate each pulse it receives with its appropriate segment, even when pulses that belong to different segments arrive from the same neighbor.

For instance, consider the node d in Figure 1, and suppose it has just started its data phase and received a clockwise DATA pulse from node c. This data pulse could have originated at node e and should be forwarded to node a, or it could have originated at node a and should be forwarded to node e.

To avoid this type of confusion, each node u tracks throughout the execution in which of its segments the token is located. Specifically, the node u maintains the invariant that $prev_{u,i}$ and $next_{u,i}$ reflect the previous and next nodes of its occurrence number i, for $0 \leq i \leq k_u - 1$, in the specific rotation of the cycle that **starts** with the token segment considered by u, i.e., the token always resides within segment 0, (locally) for all nodes. To achieve this, node u applies a local rotation function upon receiving information about the token holder, namely, upon receiving a TOKEN pulse which we show that can be traced correctly to a specific segment.

Challenge 2: Distinguishing between different DATA pulses. Another challenge that arises is how to distinguish between different DATA pulses. Recall that in the simulator for the simple cycle, the d DATA pulses are forwarded concurrently, in the sense that the token holder issues all d DATA pulses and only then waits to receive them. However, once an edge appears more than once in C, its endpoint u needs to tell apart the case in which it receives two different DATA pulses on that edge from the case in which it receives the same DATA pulse on that edge but from different segments. This is crucial because d is not known in advance (and in fact the value of d is the exact piece of information that needs to be learned).

We overcome this challenge by making sure that the DATA pulses get forwarded in a sequential manner as follows. The node-occurrence that is the token holder does not issue all d DATA pulses, but rather waits to receive DATA pulse number ℓ from its counter-clockwise neighbor before issuing DATA pulse $\ell+1$, for $1 \leq \ell \leq d-1$. A node u that receives the DATA pulse for the i-th time since the last reception of a counterclockwise END pulse, forwards it to $next_{u,i-1}$ (where the index is taken mod k_u).

Challenge 3: REQUEST pulses have no guaranteed structure. While our approach for overcoming Challenges 1 and 2 allows the nodes to have consistent rotations of the cycle and the token segments for streamlining the DATA, END, and TOKEN pulses, it is insufficient for handling REQUEST pulses. This is because each of the other three types of pulses traverses the cycle sequentially (or partially traverses in case of a TOKEN pulse), but REQUEST pulses could be initiated by different nodes, so that a node that receives a REQUEST pulse does not have any particular promise about its origin and hence cannot tell which neighbor to forward this pulse to.

We remedy this uncertainty by having each node disseminate REQUEST pulses to all of its clockwise neighbors, regardless of their origin (which is not known to the node). We show that in the case of REQUEST pulses, this coarse action satisfies the conditions that are needed in order for the simulator to work correctly, despite its somewhat more aggressive and unstructured nature.

Acknowledgements. This project has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement no. 755839. Ran Gelles is supported in part by the Israel Science Foundation (ISF) through Grant No. 1078/17 and the United States-Israel Binational Science Foundation (BSF) through Grant No. 2020277. Gal Sela is supported in part by the Israel Science Foundation (ISF) through Grant No. 1102/21.

REFERENCES

[1] Abhinav Aggarwal, Varsha Dani, Thomas P. Hayes, and Jared Saia. 2018. Sending a Message with Unknown Noise. In *ICDCN*. Article 8, 10 pages.
[2] Abhinav Aggarwal, Varsha Dani, Thomas P. Hayes, and Jared Saia. 2020. A Scalable Algorithm for Multiparty Interactive Communication with Private Channels. In *ICDCN*. Article 8, 15 pages.
[3] Noga Alon, Mark Braverman, Klim Efremenko, Ran Gelles, and Bernhard Haeupler. 2019. Reliable communication over highly connected noisy networks. *Distributed Computing* 32, 6 (2019), 505–515.
[4] Michael Barborak, Anton Dahbura, and Miroslaw Malek. 1993. The consensus problem in fault-tolerant computing. *ACM Computing Surveys (CSur)* 25, 2 (1993), 171–220.
[5] Martin Biely. 2003. An optimal Byzantine agreement algorithm with arbitrary node and link failures. In *Fifteenth IASTED International Conference on Parallel and Distributed Computing and Systems*, Vol. 1. 146–151.
[6] Mark Braverman, Klim Efremenko, Ran Gelles, and Bernhard Haeupler. 2017. Constant-Rate Coding for Multiparty Interactive Communication Is Impossible. *J. ACM* 65, 1, Article 4 (2017), 41 pages.
[7] Keren Censor-Hillel, Shir Cohen, Ran Gelles, and Gal Sela. 2022. Distributed Computations in Fully-Defective Networks. arXiv:2205.11148
[8] Keren Censor-Hillel, Ran Gelles, and Bernhard Haeupler. 2019. Making asynchronous distributed computations robust to noise. *Distributed Computing* 32, 5 (2019), 405–421.
[9] Varsha Dani, Mahnush Movahedi, Jared Saia, and Maxwell Young. 2015. Interactive Communication with Unknown Noise Rate. In *ICALP*. 575–587.
[10] Pallab Dasgupta. 1998. Agreement under faulty interfaces. *Inform. Process. Lett.* 65, 3 (1998), 125–129.
[11] Danny Dolev. 1982. The Byzantine generals strike again. *Journal of Algorithms* 3, 1 (1982), 14–30.
[12] Elena Dubrova. 2013. *Fault-Tolerant Design*. Springer.
[13] Klim Efremenko, Elad Haramaty, and Yael Tauman Kalai. 2020. Interactive Coding with Constant Round and Communication Blowup. In *ITCS*. 7:1–7:34.
[14] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. 1985. Impossibility of Distributed Consensus with One Faulty Process. *J. ACM* 32, 2 (1985), 374–382.
[15] Ran Gelles. 2017. Coding for Interactive Communication: A Survey. *Foundations and Trends® in Theoretical Computer Science* 13, 1–2 (2017), 1–157.
[16] Ran Gelles and Siddharth Iyer. 2020. Interactive Coding Resilient to an Unknown Number of Erasures. In *OPODIS*. 13:1–13:16.
[17] Ran Gelles and Yael T. Kalai. 2019. Constant-Rate Interactive Coding Is Impossible, Even in Constant-Degree Networks. *IEEE Transactions on Information Theory* 65, 6 (2019), 3812–3829.
[18] Ran Gelles, Yael Tauman Kalai, and Govind Ramnarayan. 2019. Efficient Multiparty Interactive Coding for Insertions, Deletions, and Substitutions. In *PODC*. 137–146.
[19] Ran Gelles, Ankur Moitra, and Amit Sahai. 2014. Efficient Coding for Interactive Communication. *IEEE Transactions on Information Theory* 60, 3 (2014), 1899–1913.
[20] Li Gong, Patrick Lincoln, and John Rushby. 1995. Byzantine agreement with authentication: Observations and applications in tolerating hybrid and link faults. In *Dependable Computing and Fault Tolerant Systems*, Vol. 10. IEEE Computer Society, 139–158.
[21] J.N. Gray. 1978. Notes on data base operating systems. In *Operating Systems*. Lecture Notes in Computer Science, Vol. 60. Springer, Berlin, Heidelberg.
[22] Yael Hitron and Merav Parter. 2021. Broadcast CONGEST Algorithms against Adversarial Edges. In *DISC*. 23:1–23:19.
[23] Yael Hitron and Merav Parter. 2021. General CONGEST Compilers against Adversarial Edges. In *DISC*. 24:1–24:18.
[24] William M. Hoza and Leonard J. Schulman. 2016. The Adversarial Noise Threshold for Distributed Protocols. In *SODA*. 240–258.
[25] Abhishek Jain, Yael Tauman Kalai, and Allison Lewko. 2015. Interactive Coding for Multiparty Protocols. In *ITCS*. 1–10.
[26] Israel Koren and C. Mani Krishna (Eds.). 2020. *Fault-Tolerant Systems* (second edition ed.). Morgan Kaufmann, San Francisco (CA).
[27] Allison Lewko and Ellen Vitercik. 2015. Balancing Communication for Multiparty Interactive Coding. arXiv:1503.06381
[28] Merav Parter and Eylon Yogev. 2019. Low Congestion Cycle Covers and Their Applications. In *SODA*. 1673–1692.
[29] Merav Parter and Eylon Yogev. 2019. Optimal Short Cycle Decomposition in Almost Linear Time. In *ICALP*. 89:1–89:14.
[30] Andrzej Pelc. 1992. Reliable communication in networks with Byzantine link failures. *Networks* 22, 5 (1992), 441–459.
[31] Kenneth J. Perry and Sam Toueg. 1986. Distributed agreement in the presence of processor and communication faults. *IEEE Transactions on Software Engineering* SE-12, 3 (1986), 477–482.
[32] Sridhar Rajagopalan and Leonard Schulman. 1994. A coding theorem for distributed computation. In *STOC*. 790–799.
[33] Michel Raynal. 2018. *Fault-Tolerant Message-Passing Distributed Systems: An Algorithmic Approach*. Springer, Cham. 459 pages.
[34] Herbert Ellis Robbins. 1939. A theorem on graphs, with an application to a problem of traffic control. *The American Mathematical Monthly* (1939).
[35] Nicola Santoro and Peter Widmayer. 1990. Distributed function evaluation in the presence of transmission faults. In *International Symposium on Algorithms*. Springer, 358–367.
[36] Hasan M. Sayeed, Marwan Abu-Amara, and Hosame Abu-Amara. 1995. Optimal Asynchronous Agreement and Leader Election Algorithm for Complete Networks with Byzantine Faulty Links. *Distributed Computing* 9, 3 (1995), 147–156.
[37] Ulrich Schmid, Bettina Weiss, and Idit Keidar. 2009. Impossibility Results and Lower Bounds for Consensus under Link Failures. *SIAM J. Comput.* 38, 5 (2009), 1912–1951.
[38] Leonard J. Schulman. 1992. Communication on noisy channels: a coding theorem for computation. In *FOCS*. 724–733.
[39] Leonard J. Schulman. 1993. Deterministic coding for interactive communication. In *STOC* (San Diego, California, United States). 747–756.
[40] Hin-Sing Siu, Yeh-Hao Chin, and Wei-Pang sm. 1998. Byzantine agreement in the presence of mixed faults on processors and links. *IEEE Transactions on Parallel and Distributed Systems* 9, 4 (1998), 335–345.
[41] Hassler Whitney. 1932. Non-Separable and Planar Graphs. *Trans. Amer. Math. Soc.* 34, 2 (1932), 339–362.